# Intro to Data Science Course Summary

Generated by Scribe.AI

December 10, 2024

## 1 Key Terms

- **Data Never Sleeps 10.0**: An infographic visualizing the massive amount of data generated every minute across various online platforms.
- **Data Science**: An interdisciplinary field that uses scientific methods, processes, algorithms, and systems to extract knowledge and insights from structured and unstructured data.
- **Data Acquisition**: The process of obtaining data from various sources, such as databases, APIs, web scraping, or sensors.
- **Data Cleaning**: The process of identifying and correcting or removing errors, inconsistencies, and inaccuracies in data.
- **Data Manipulation**: The process of transforming and modifying data to make it suitable for analysis.
- **Data Wrangling**: The process of cleaning, structuring, and transforming raw data into a usable format for analysis.
- **Data Visualization**: The graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.
- **Data Management**: The development and execution of architectures, policies, and procedures to manage data assets throughout their lifecycle.
- **Big Data Systems**: Systems designed to handle and process extremely large and complex datasets that exceed the capacity of traditional data processing systems.
- **Hypothesis Testing**: A statistical method used to determine whether there is enough evidence in a sample of data to infer that a certain condition is true for the population as a whole.
- **A/B Testing**: A randomized experiment with two variants, A and B, where variant A is typically the control and variant B is the variation.
- **Correlation vs. Causation**: Correlation refers to a statistical relationship between two or more variables, while causation implies that one variable directly influences another.
- **Similarity/Distance Measures**: Quantitative measures used to assess the similarity or dissimilarity between data points or objects.
- **Clustering Methods**: Unsupervised machine learning techniques used to group similar data points together into clusters.
- **Dimensionality Reduction**: Techniques used to reduce the number of variables in a dataset while preserving important information.
- **Collaborative Filtering**: A technique used in recommender systems to predict user preferences based on the preferences of similar users.

- **Data Engineering**: The process of designing, building, and maintaining systems for collecting, storing, processing, and analyzing data.
- **SQL**: A domain-specific language used for managing and manipulating data in relational database management systems.
- **MapReduce**: A programming model for processing large datasets across a cluster of computers.
- **Spark**: A fast, general-purpose cluster computing system for large-scale data processing.
- **Hadoop**: An open-source framework for storing and processing large datasets across clusters of computers.
- **Ethics in Data Science**: The principles and guidelines that govern the responsible and ethical use of data and data science techniques.
- **Palindrome**: A sequence that reads the same backward as forward.
- **Jupyter Notebook**: An open-source web application that allows you to create and share documents that contain live code, equations, visualizations, and narrative text.
- **Python**: A high-level, general-purpose programming language widely used in data science for its versatility and extensive libraries.
- **Pandas**: A Python library providing high-performance, easy-to-use data structures and data analysis tools.
- **NumPy**: A Python library providing support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays.
- **IPython**: An interactive computing environment for Python that provides enhanced features for interactive work, including improved shell, introspection, and debugging.
- **Data Never Sleeps 4.0**: An infographic showcasing the massive amount of data generated every minute across various online platforms.
- **What Wal-Mart Knows About Customers' Habits**: A New York Times article illustrating how Wal-Mart used data analysis to predict customer behavior during Hurricane Frances.
- **Big Data**: Extremely large and complex datasets that require specialized tools and techniques for analysis and management.
- **Data mining**: The process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data.
- **Machine learning**: The development of computer systems that automatically improve with experience.
- **Modern Data Scientist**: A professional possessing a blend of mathematical, statistical, programming, database, domain, soft, communication, and visualization skills.
- **Fraud Prediction**: The use of machine learning to identify potentially fraudulent activities, such as in the stock market.
- **Recommender Systems**: Systems that predict user preferences and provide personalized recommendations, as exemplified by the Netflix Prize competition.
- **Sky Image Cataloging**: The use of data analysis to automatically categorize astronomical objects in large datasets of images.
- **Detection of early Alzheimer's**: Using machine learning techniques, such as support vector machines, to identify early signs of Alzheimer's disease from MRI scans.

- **Early identification of heart failure**: Utilizing deep learning models to predict heart failure months before traditional methods.
- **Early detection of sepsis**: Employing machine learning models, such as Cox proportional hazards models, to predict septic shock earlier than traditional methods.
- **Personalized autism treatments**: Using reinforcement learning to optimize treatment strategies for autistic children based on individual needs.
- **How Companies Learn Your Secrets**: An exploration of how companies utilize data to understand and predict customer behavior, often for targeted marketing.
- **BIG DATA ANALYTICS PIPELINE \& STAKEHOLDERS**: A visual representation of the big data analytics pipeline, highlighting the different stages and the roles of various stakeholders.
- **The Data Mining Process**: A structured approach to extracting meaningful patterns and insights from data, illustrated with a Target example.
- **What you want to do affects what you need**: A statement highlighting the relationship between the desired outcome of a data analysis task and the appropriate techniques to employ.
- **Python 3**: A high-level, general-purpose programming language widely used in data science for its readability and extensive libraries.
- **Python Notebooks**: Interactive coding environments that allow for combining code, text, and visualizations in a single document.
- **Anaconda**: A popular Python distribution that simplifies the installation and management of Python packages and environments.
- **Interpreted Language**: A programming language where code is executed line by line without prior compilation.
- **Dynamically Typed**: A programming language where variable types are checked during runtime, not during compilation.
- **Lists**: Ordered, mutable sequences of items in Python. They can contain elements of different data types.
- **Tuples**: Ordered, immutable sequences of items in Python. They can contain elements of different data types.
- **Dictionaries**: Unordered collections of key-value pairs in Python. Keys must be immutable, while values can be of any type.
- **Indentation**: In Python, indentation is used to define code blocks, unlike other languages that use braces.
- **Immutable**: An object whose state cannot be modified after it is created.
- **Mutable**: An object whose state can be modified after it is created.
- **Slicing**: A technique for extracting a portion of a sequence (like a list or string) in Python using array notation.
- **‘in‘ operator**: A Python operator used to check if a value exists within a sequence or a substring exists within a string.
- **‘+‘ operator**: In Python, the ‘+‘ operator can be used to concatenate strings and lists.
- **‘append()‘ method**: A Python list method that adds an element to the end of a list.
- **‘insert()‘ method**: A Python list method that inserts an element at a specified index in a list.

- **‘extend()‘ method**: A Python list method that adds all items from an iterable (like another list) to the end of a list.
- **‘index()‘ method**: A Python list method that returns the index of the first occurrence of a specified element.
- **‘count()‘ method**: A Python list method that returns the number of times a specified element appears in a list.
- **‘remove()‘ method**: A Python list method that removes the first occurrence of a specified element from a list.
- **‘reverse()‘ method**: A Python list method that reverses the order of elements in a list in-place.
- **‘sort()‘ method**: A Python list method that sorts the elements of a list in-place.
- **‘del‘ keyword**: A Python keyword used to delete variables or elements from data structures like lists and dictionaries.
- **‘keys()‘ method**: A Python dictionary method that returns a view object containing the keys of the dictionary.
- **‘values()‘ method**: A Python dictionary method that returns a view object containing the values of the dictionary.
- **‘items()‘ method**: A Python dictionary method that returns a view object containing the key-value pairs of the dictionary as tuples.
- **‘upper()‘ method**: A Python string method that converts a string to uppercase.
- **‘lower()‘ method**: A Python string method that converts a string to lowercase.
- **‘strip()‘ method**: A Python string method that removes leading and trailing whitespace from a string.
- **‘def‘ keyword**: A Python keyword used to define a function.
- **‘return‘ keyword**: A Python keyword used to return a value from a function.
- **‘None‘**: A special constant in Python representing the absence of a value.
- **‘self‘**: A special parameter in Python methods that refers to the instance of the class.
- **‘___init___‘**: A special method in Python classes that serves as the constructor.
- **‘open()‘ function**: A Python function used to open files.
- **‘read()‘ method**: A Python file method used to read the entire content of a file into a string.
- **‘csv‘ module**: A Python module used for reading and writing CSV files.
- **‘sys‘ module**: A Python module that provides access to system-specific parameters and functions.
- **‘sys.argv‘**: A list in the ‘sys‘ module containing command-line arguments passed to a Python script.
- **Attributes**: A property or characteristic of an entity (e.g., eye color, temperature).
- **Entities**: A collection of attributes.
- **Tabular Data**: Simple structured data organized in rows and columns.
- **Structured Data (Relational)**: A collection of data items with a defined schema and relationships between them, such as relational databases or graph data.

- **Semi-Structured Data**: Data with less formal structure than relational data but with tags or markers to delineate semantic elements, such as XML or JSON.
- **Unstructured Data**: Data that is not organized according to a predefined structure, such as free text, videos, photos, music, and messages.
- **CSV Files**: Delimited text files where data is separated by commas, tabs, or spaces.
- **JSON Files**: A way of encapsulating Javascript objects; data looks much like a dictionary in Python, with keys and values stored.
- **XML/HTML Files**: Markup languages used to structure content on the web; XML is hierarchical with tags, while HTML is similar but often lacks closed tags and focuses on appearance.
- **RegEx**: A sequence of characters that define a search pattern.
- **'re' module**: A Python module providing support for regular expressions.
- **Raw String**: A string literal prefixed with 'r' in Python, preventing backslash escapes from being interpreted.
- **'re.compile()'**: A function in Python's 're' module that compiles a regular expression pattern for reuse.
- **'re.match()'**: A function in Python's 're' module that tests if a regular expression matches at the beginning of a string.
- **'re.search()'**: A function in Python's 're' module that scans a string for the first occurrence of a regular expression pattern.
- **'re.findall()'**: A function in Python's 're' module that finds all non-overlapping matches of a regular expression pattern in a string.
- **'re.finditer()'**: A function in Python's 're' module that finds all non-overlapping matches of a regular expression pattern in a string, returning an iterator of match objects.
- **'match.group()'**: A method of a match object in Python's 're' module that returns the matched substring.
- **'match.span()'**: A method of a match object in Python's 're' module that returns a tuple containing the start and end indices of the matched substring.
- **Quantifiers**: Symbols in regular expressions that specify how many times a character or group should appear in a match (e.g., '¿, '', '+', 'n').
- **Greedy Matching**: The default behavior of regular expressions, where the engine tries to match as much text as possible.
- **Lazy Matching**: A matching behavior in regular expressions, where the engine tries to match as little text as possible, achieved using the '¿ quantifier.
- **Capturing Groups**: Portions of a regular expression enclosed in parentheses, allowing access to specific parts of the matched string.
- **Beautiful Soup**: A Python library for parsing HTML and XML documents.
- **'soup.prettify()'**: A method in Beautiful Soup that formats the parsed HTML for better readability.
- **'soup.find_all()'**: A method in Beautiful Soup that finds all tags matching a specified criteria.
- **'link.get_text()'**: A method in Beautiful Soup that extracts the text content of a tag.
- **'p.find()'**: A method in Beautiful Soup that searches for a tag within a given tag.

- **TF-IDF**: Term Frequency-Inverse Document Frequency, a numerical statistic that reflects how important a word is to a document in a collection or corpus.
- **Term Frequency (TF)**: The number of times a word appears in a document, divided by the total number of words in that document.
- **Inverse Document Frequency (IDF)**: A measure of how much information the word provides, calculated as $ln(\frac{N}{1+n_t})$, where N is the total number of documents and $n_t$ is the number of documents containing the term.
- **Document-Term Matrix**: A matrix where rows represent documents, columns represent unique words, and cells contain the frequency of each word in each document.
- **Stop Words**: Common words (like "the," "a," "is") that are often removed from text data before analysis.
- **Stemming**: Reducing words to their root form (e.g., "running" to "run").
- **Data Organization**: The structured arrangement of data for efficient storage, retrieval, and analysis in a data science context.
- **Augmenting Data**: The process of adding new data or features to an existing dataset, such as adding new columns or rows.
- **Subsetting Data**: Selecting a portion of a dataset based on specific criteria, such as filtering rows based on column values or selecting specific columns.
- **Cleaning Data**: The process of identifying and correcting or removing errors, inconsistencies, and inaccuracies in a dataset, such as handling missing values, removing duplicates, or correcting typos.
- **Missing Data**: The absence of values in a dataset, which can be handled through various techniques such as imputation, deletion, or using default values.
- **Transformation**: The process of modifying or converting data from one form to another, such as scaling values, normalizing data, or converting data types.
- **Aggregating Data**: The process of summarizing data by grouping it into categories and applying summary functions, such as calculating the mean, sum, or count for each group.
- **Combining Data**: The process of integrating data from multiple sources into a single dataset, such as concatenating or merging DataFrames.
- **Data Munging Outside of Python/Pandas**: Data manipulation techniques that utilize command-line tools and shell scripting, often for tasks involving text processing and data transformation.
- **Anscombe's Quartet**: Four datasets with identical summary statistics (mean, variance, correlation, regression line) but vastly different distributions, highlighting the importance of data visualization.
- **Strategic communication**: Communication that starts with a message and then seeks information to support it.
- **Candid communication**: Communication that starts with information and analyzes it to discover messages, potentially refuting initial assumptions.
- **Chart**: A visual representation of data encoded using shapes, colors, or proportions.
- **Map**: A visual depiction of a geographical area with overlaid data relevant to that area.
- **Infographic**: A multi-section visual representation designed to convey one or more specific messages.
- **Playfair charts**: Early examples of data visualization, showcasing historical data representation through line and bar graphs.

- **Minard's visualization**: A sophisticated historical visualization combining map and line graphs to depict the French army's movement and losses during the 1812-1813 Russian campaign.
- **Spurious Correlations**: Correlations between variables that appear statistically significant but lack a causal relationship.
- **Hockey stick chart**: A line graph showing a long-term temperature trend with a sharp upward turn in recent decades, illustrating climate change.
- **Exploratory Data Analysis**: The process of analyzing data to summarize its main characteristics, often with visual methods, to gain an understanding of the data before formal modeling.
- **Principal Component Analysis (PCA)**: A dimensionality reduction technique that transforms a dataset into a new set of uncorrelated variables, called principal components, which capture the maximum variance in the data.
- **Multidimensional scaling**: A dimensionality reduction technique used to visualize the distances between objects in a high-dimensional space in a lower-dimensional space, typically two or three dimensions.
- **Measures of location**: Statistical measures that describe the central tendency of a dataset, such as the mean, median, quartiles, and mode.
- **Measures of dispersion or variability**: Statistical measures that describe the spread or variability of a dataset, such as the variance, standard deviation, range, and skew.
- **'matplotlib.pyplot.scatter'**: A function in the Matplotlib library used to create scatter plots in Python. It takes x and y coordinates as input and allows for customization of color, marker style, and label.
- **'\%matplotlib inline'**: A Jupyter Notebook magic command that configures Matplotlib to display plots directly within the notebook.
- **matplotlib.pyplot**: A collection of functions that make matplotlib work like MATLAB, providing a convenient way to create plots and visualizations.
- **'plt.scatter'**: A Matplotlib function used to create scatter plots.
- **'matplotlib.pyplot.subplots'**: A Matplotlib function that creates a figure and a set of subplots.
- **'ax.scatter'**: A Matplotlib function to create a scatter plot on a specific axes object.
- **'ax.set_title'**: A Matplotlib function to set the title of a subplot.
- **'ax.set_xlim'**: A Matplotlib function to set the x-axis limits of a subplot.
- **'ax.set_ylim'**: A Matplotlib function to set the y-axis limits of a subplot.
- **'fig.tight_layout'**: A Matplotlib function to automatically adjust subplot parameters for a tight layout.
- **Scatter plot matrix**: A grid of scatter plots showing the pairwise relationships between multiple variables in a dataset.
- **Jitter points**: A technique used in data visualization to reduce overplotting by adding small amounts of random noise to the data points.
- **Contour plot**: A 2D representation of a 3D surface, showing lines of constant z-values.
- **Line plot**: A plot that displays data as a series of points connected by line segments.
- **'plt.plot'**: A Matplotlib function used to create line plots.
- **Histogram**: A graphical representation of the distribution of numerical data.

- **‘plt.hist‘**: A Matplotlib function used to create histograms.
- **‘bins‘**: A parameter in ‘plt.hist‘ that specifies the number of bins in a histogram.
- **‘rwidth‘**: A parameter in ‘plt.hist‘ that specifies the relative width of the bars in a histogram.
- **‘density‘**: A parameter in ‘plt.hist‘ that specifies whether to normalize the histogram to represent probability density.
- **Smoothed density plot**: A plot that shows the probability density of a continuous variable using a smoothed curve.
- **Kernel density estimation**: A non-parametric way to estimate the probability density function of a random variable.
- **Bar plot**: A chart that uses bars to represent the frequencies or values of different categories.
- **‘plt.bar‘**: A Matplotlib function used to create bar plots.
- **Stacked bar chart**: A bar chart where bars are stacked on top of each other to show the contribution of different components to a total.
- **Grouped bar chart**: A bar chart where bars are grouped together to compare different categories or variables.
- **Box plot**: A visualization that displays the distribution of a dataset through its quartiles.
- **‘plt.boxplot‘**: A Matplotlib function used to create box plots.
- **Violin plot**: A combination of a box plot and a kernel density estimation plot.
- **‘ax.violinplot‘**: A Matplotlib function used to create violin plots.
- **‘plt.GridSpec‘**: A Matplotlib function used to create a grid of subplots.
- **ggplot**: A data visualization system based on the grammar of graphics.
- **plotnine**: A Python implementation of the grammar of graphics.
- **‘ggplot(data=iris) + geom_point(...)‘**: A plotnine command to create a scatter plot of petal length vs. sepal length from the iris dataset.
- **‘aes‘**: An aesthetic mapping function in plotnine.
- **‘geom_point‘**: A plotnine geometry function to create scatter plots.
- **‘geom_smooth‘**: A plotnine geometry function to add a smoothed line to a plot.
- **‘geom‘**: A plotnine geometry function that specifies the type of graphical object to use in a plot.
- **‘stat‘**: A plotnine statistical transformation function that transforms the data before plotting.
- **‘scale‘**: A plotnine scale function that controls the mapping between data values and visual properties.
- **‘coord‘**: A plotnine coordinate system function that controls the coordinate system of a plot.
- **‘facet‘**: A plotnine faceting function that creates subplots based on different values of a variable.
- **‘position‘**: A plotnine position adjustment function that controls the position of overlapping geoms.
- **‘theme‘**: A plotnine theme function that controls the overall appearance of a plot.
- **Plotly**: A library for creating interactive plots.

- **'go.Scatter'**: A Plotly trace object used to create scatter plots and line plots.
- **'go.Bar'**: A Plotly trace object used to create bar charts.
- **'go.Heatmap'**: A Plotly trace object used to create heatmaps.
- **'go.Histogram'**: A Plotly trace object used to create histograms.
- **'go.Box'**: A Plotly trace object used to create box plots.
- **'plotly.express'**: A high-level interface to Plotly for creating plots easily.
- **'px.box'**: A plotly.express function used to create box plots.
- **Data Integrity**: The accuracy and reliability of the data used in a visualization.
- **Clear and Concise**: The ease with which a visualization can be understood.
- **Appropriate Choice of Visualization**: Selecting the visualization type that best suits the data and the message.
- **Context and Story**: Providing background information and framing the visualization within a narrative.
- **Accessibility and Inclusivity**: Designing visualizations that are accessible to a wide audience, including those with visual impairments.
- **Interactivity and Exploration**: Incorporating interactive elements to allow users to explore the data further.
- **Visual Appeal and Aesthetics**: Making the visualization visually appealing and aesthetically pleasing.
- **Visual Estimates**: The ability of users to accurately estimate values from a visualization.
- **Baselines and scales of plots**: The choice of baseline and scale for the axes of a plot.
- **Data quality**: The overall fitness of data for its intended uses. This includes considerations of accuracy, completeness, consistency, and timeliness.
- **Noise**: Measurement error in data values, including random errors (inconsistencies) and systematic (bias) errors (consistent deviations in one direction).
- **Outliers**: Data objects considerably different from most other data objects in a dataset; they may indicate interesting cases or errors.
- **Missing values**: Data values that are absent from a dataset due to various reasons, such as information not being collected or attributes not being applicable.
- **Duplicate data**: Data entities that are duplicates or near-duplicates of one another, often arising from merging data from different sources.
- **The Truth Continuum**: A concept representing the range of accuracy and reliability in data, from completely true to completely false.
- **Patternicity bug**: The tendency to perceive patterns in random data, leading to misinterpretations.
- **Storytelling bug**: The tendency to create coherent narratives to explain patterns in data, even if those narratives are not supported by evidence.
- **Confirmation bug**: The tendency to favor evidence that confirms pre-existing beliefs and to disregard contradictory evidence.
- **Heuristics and Biases**: Cognitive shortcuts and systematic errors in thinking that can lead to inaccurate judgments and decisions.

- **Availability heuristic**: The tendency to overestimate the likelihood of events that are easily recalled, often due to their salience or recent occurrence.
- **Representativeness heuristic**: The tendency to judge the probability of an event based on how similar it is to a prototype or stereotype.
- **Gambler's fallacy**: The mistaken belief that past events can influence the probability of future independent events.
- **Confirmation bias**: The tendency to search for, interpret, favor, and recall information in a way that confirms or supports one's prior beliefs or values.
- **Explaining Patterns in Data**: A good explanation involves a claim answering a question, evidence from data supporting the claim, and reasoning connecting the evidence to the claim.
- **Claims**: Assertions that may or may not be supported by evidence. Claims should be evaluated critically using data and reasoning.
- **Asking Scientific Questions**: Formulating questions that can be answered using data analysis, often involving comparisons or the investigation of causal relationships.
- **Sample Statistics**: Numerical characteristics of a sample drawn from a larger population. Sample statistics are used to estimate population parameters, but they are subject to sampling error.
- **One Sample T-Test**: A statistical test used to determine if the mean of a single group is significantly different from a known or hypothesized mean.
- **Two Sample T-Test**: A statistical test used to determine if the means of two groups are significantly different. This can be an independent samples t-test or a paired t-test.
- **Paired T-Test**: A type of two-sample t-test used when the observations in the two groups are paired, such as before-and-after measurements on the same subjects.
- **Two-sided vs. One-sided Tests**: Two-sided tests assess whether a parameter is different from a hypothesized value, while one-sided tests assess whether a parameter is greater than or less than a hypothesized value.
- **Statistical Power**: The probability that a hypothesis test will correctly reject a false null hypothesis. Low power can lead to false negatives (Type II errors).
- **How to Increase Statistical Power**: Methods to increase the likelihood of detecting a true effect, including increasing sample size, decreasing sample variability, increasing effect size, and increasing the significance level ().
- **Sampling Distribution**: The probability distribution of a statistic (e.g., mean) when calculated from a random sample of size $n$.
- **Standard Error**: The standard deviation of the sampling distribution.
- **Margin of Error**: The confidence interval of the sampling distribution.
- **Confidence Interval**: An expression of uncertainty with respect to an estimated statistic.
- **Sampling**: The main technique employed for data selection, often used for preliminary investigation and final data analysis.
- **Simple Random Sampling**: A sampling method where there is an equal probability of selecting any particular item.
- **Sampling Without Replacement**: A sampling method where items are removed from the population as they are selected.

- **Sampling With Replacement**: A sampling method where items are not removed from the population; the same item can be selected multiple times.
- **Stratified Sampling**: A sampling method where the data is split into several partitions, and then random samples are drawn from each partition.
- **Representative Sample**: A sample that has approximately the same properties (of interest) as the original data set.
- **Simpson's Paradox**: A phenomenon where groups of data show one trend, but this trend is reversed when the groups are combined.
- **Counterfactuals**: The hypothetical outcome that would have occurred if a treatment or intervention had not been applied.
- **Randomization**: A method of assigning subjects to groups to minimize selection bias and allow for causal inference.
- **Hierarchical clustering**: Clusters form a hierarchy, computed bottom-up or top-down.
- **Flat clustering**: No inter-cluster structure.
- **Hard clustering**: Items are assigned to a single unique cluster.
- **Soft clustering**: Items are assigned to one or more clusters; membership is often expressed as a real-valued function.
- **K-means clustering**: One of the most commonly used clustering algorithms; partitions objects into $K$ disjoint subsets; assumes objects are $p$-dimensional vectors and uses a distance measure (typically Euclidean distance).
- **Euclidean distance**: The most commonly used distance measure for continuous data; $d_E(x, y) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$, where $p$ is the number of attributes/dimensions.
- **Proximity**: A general term indicating similarity and dissimilarity.
- **Distance**: A measure of dissimilarity.
- **Hamming distance**: Used for bit/binary vectors; counts the number of bits that differ; can be used more generally for discrete data to count matches.
- **Matching coefficient**: Hamming distance normalized by the number of attributes/bits.
- **Inertia score**: In K-means clustering, the sum of squared distances from each point to its cluster centroid.
- **Purity**: A measure of the extent to which clusters contain objects of a particular class; $purity(C, G) = \frac{1}{N}\sum_{i=1}^{K} \max_j N_j \in G_i$.
- **Personas**: Representations of target users to provide a common understanding across all teams.
- **Zettabytes**: A unit of information equal to $10^{21}$ bytes.
- **Petabytes**: A unit of information equal to $10^{15}$ bytes.
- **Terabytes**: A unit of information equal to $10^{12}$ bytes.
- **Gigabytes**: A unit of information equal to $10^9$ bytes.

- **Mean Time Between Failures (MTBF)**: The average time between failures of a system.
- **Message Passing Interface (MPI)**: A standardized and portable message-passing system used for parallel computing.
- **Inverted Index**: A data structure that stores a mapping from words to the documents containing them.
- **Data Locality**: The principle of performing computations on the same machine where the data resides.
- **Namenode**: The central server in HDFS that manages the metadata of the file system.
- **Datanodes**: The storage nodes in HDFS that store the actual data blocks.
- **Matrix Factorization**: A technique used to decompose a high-dimensional data matrix into a product of two lower-dimensional matrices, revealing latent factors that capture underlying structure in the data.
- **Embeddings**: A vector of latent factor values that projects an item (e.g., movie) into a lower-dimensional space.
- **Latent Factors**: Underlying, hidden variables that explain the relationships between observed variables in a dataset. In the context of recommender systems, these factors represent user preferences and item characteristics.
- **Low Rank Matrix Approximations**: Approximating a high-rank matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of a prediction model by calculating the square root of the average of the squared differences between predicted and actual values.
- **Netflix Prize**: A competition launched by Netflix in 2006 that challenged participants to improve the accuracy of its movie recommendation system by 10%.
- **Disparate Treatment**: A type of discrimination where decisions are (partly) based on a subject's protected attribute information.
- **Disparate Impact**: A type of discrimination where outcomes disproportionally hurt (or benefit) people with certain sensitive attribute values.

## 2 Problem Types

- **Data Volume and Variety**: The sheer volume and variety of data generated daily across various online platforms is highlighted, emphasizing the scale of data in the modern digital world.
- **Data Acquisition, Cleaning, and Manipulation**: The course will cover methods for acquiring data (e.g., web crawling), processing, parsing, manipulating, and cleaning data.
- **Data Visualization and Communication**: The importance of visualizing data effectively and communicating results clearly is stressed. Basic plotting techniques in Python will be taught.
- **Statistical Inference and Hypothesis Testing**: The course will introduce statistical inference, hypothesis testing, A/B testing, and the distinction between correlation and causation.
- **Clustering and Dimensionality Reduction**: The course will cover common similarity/distance measures, basic clustering methods, and dimensionality reduction techniques.
- **Recommender Systems**: The course will explore recommender systems and collaborative filtering methods.
- **Large-Scale Data Analysis**: The course will discuss data engineering, databases (SQL), mapReduce processing, Spark, and Hadoop.

- **Ethical Considerations in Data Science**: The course will address ethical issues related to privacy, fairness, and bias in data science.
- **Palindrome String Detection**: A programming assignment requires writing a Python function to determine if a string is a palindrome.
- **Predicting Customer Behavior Based on Data**: Wal-Mart used data analysis to predict customer purchasing habits before Hurricane Frances, anticipating demand for unusual items like Pop-Tarts and beer.
- **Understanding Big Data's Impact on Businesses**: This problem explores the various applications of big data across different industries, highlighting its use in customer analytics, operational analytics, and new product innovation. The economic impact and market size of big data are also discussed.
- **Applying Data Mining and Machine Learning Techniques**: This problem introduces the concepts of data mining (identifying patterns in data) and machine learning (building systems that improve with experience), emphasizing their importance in data science.
- **Defining the Skillset of a Modern Data Scientist**: This problem outlines the diverse skillset required for a modern data scientist, encompassing math and statistics, programming and database management, domain knowledge, and communication skills.
- **Analyzing Historical Developments in Data Science**: This problem examines the historical evolution of various fields contributing to data science, including computer science, data technology, visualization, mathematics/operations research, and statistics.
- **Using Data Science for Fraud Detection**: This problem focuses on using machine learning methods to predict fraudulent activities, specifically in the context of stock brokers on the NASDAQ.
- **Improving Recommender Systems through Data Analysis**: This problem discusses the Netflix Prize competition, highlighting the challenge of improving movie rating predictions using large datasets and ensemble methods.
- **Applying Data Science to Astronomical Data Analysis**: This problem describes the use of data science techniques to categorize stellar objects in a large dataset of sky images from the Palomar Observatory.
- **Utilizing Machine Learning for Medical Diagnosis**: This problem explores the application of machine learning to detect early-stage Alzheimer's disease using MRI images and to predict heart failure using electronic health records.
- **Analyzing User Behavior for Improved User Experience**: This problem focuses on using data analysis to understand user behavior and create user personas, which can inform the design of user interfaces and products.
- **Applying Data Science to Early Sepsis Detection**: This problem discusses the use of machine learning to predict septic shock using electronic health records, highlighting the potential for improved early detection.
- **Using Data Science for Personalized Medical Treatments**: This problem explores the use of reinforcement learning to optimize personalized autism treatments based on patient responses.
- **Understanding How Companies Use Data to Learn Customer Secrets**: This problem examines how companies, such as Target, use data to understand customer habits and tailor marketing campaigns.
- **Defining the Data Science Process**: This problem focuses on the overall data science process, from data acquisition and cleaning to model building and interpretation.

- **Identifying Appropriate Data Science Techniques for Specific Tasks**: This problem explores the various data science techniques applicable to different tasks, such as classification, regression, clustering, and link prediction.
- **Setting up a Python Environment**: This problem involves setting up the necessary Python environment, including installing Python 3, Anaconda, and Jupyter Notebook, to facilitate data science tasks.
- **Understanding Python Basics**: This problem focuses on reviewing fundamental Python programming concepts, such as data types, operators, control flow, and functions, which are essential for data science programming.
- **Working with Python Data Structures**: This problem involves understanding and utilizing Python's built-in data structures, including lists, tuples, dictionaries, and strings, for efficient data manipulation and storage.
- **File Processing in Python**: This problem involves learning how to read and process data from files, including basic text files and CSV files, using Python's built-in functions and modules.
- **Understanding Object-Oriented Programming in Python**: This problem focuses on grasping the object-oriented programming paradigm in Python, including defining classes, methods, and utilizing the 'self' keyword for object instantiation and manipulation.
- **Defining Data**: This problem involves defining and differentiating between structured, semi-structured, and unstructured data, providing examples of each type and discussing their implications for data analysis.
- **Data Acquisition and Handling**: This problem focuses on obtaining data from various sources, including CSV, JSON, and XML files, and using Python libraries like Pandas and BeautifulSoup to parse and manipulate this data.
- **Working with Pandas DataFrames**: This problem involves understanding and utilizing Pandas DataFrames for data analysis, including creating DataFrames, accessing data using 'loc' and 'iloc', performing calculations, and summarizing data using descriptive statistics.
- **String Pattern Matching with Regular Expressions**: This problem focuses on using regular expressions to identify and extract specific patterns from text data. The Python 're' module is used to implement these operations, including methods like 'match()', 'search()', 'findall()', and 'finditer()'. The concepts of greedy and non-greedy matching, as well as the use of quantifiers and capturing groups, are also explored.
- **Web Page Parsing with Beautiful Soup**: This problem involves using the Beautiful Soup library in Python to extract information from web pages. The process includes downloading web pages, parsing HTML content, and extracting specific elements like links and paragraphs. The code examples demonstrate how to navigate the parsed HTML structure to extract relevant data.
- **TF-IDF Calculation and Application**: This problem involves understanding and applying the TF-IDF (Term Frequency-Inverse Document Frequency) method to analyze text data. The concepts of Term Frequency (TF) and Inverse Document Frequency (IDF) are explained, along with their formulas. The creation of a Document-Term Matrix is also discussed as a key step in this process. The application of TF-IDF is illustrated through the analysis of State of the Union addresses.
- **Data Cleaning**: Identifying and correcting errors, inconsistencies, and missing values in a dataset. Examples include handling duplicates in an employee dataset and replacing missing values in weather data .
- **Data Manipulation**: Transforming and preparing data for analysis. Examples include renaming columns , sorting data , adding calculated columns , adding rows , deleting columns , filtering data , and handling missing data .

- **Data Aggregation**: Grouping and summarizing data to gain insights. Examples include calculating summary statistics , using the Split/Apply/Combine paradigm , and applying summary functions to grouped data .
- **Data Transformation**: Changing the format or structure of data. Examples include normalizing data , binning data , and converting numerical grades to letter grades .
- **Data Combination**: Merging and concatenating data from different sources. Examples include concatenating DataFrames , performing different types of joins (inner, outer, left, right) , and handling one-to-one, many-to-one, and one-to-many relationships .
- **Command-Line Data Processing**: Using command-line tools for data manipulation. Examples include using ‘wc‘, ‘head‘, ‘tail‘, ‘sort‘, ‘cat‘, ‘uniq‘ , ‘grep‘ , ‘sed‘ , and ‘awk‘ for data cleaning and transformation.
- **Evaluating the Effectiveness of Visualizations**: This problem focuses on assessing the quality of data visualizations, considering aspects like truthfulness, functionality, beauty, insightfulness, and enlightenment. The Anscombe’s Quartet and the “hockey stick“ chart are used as examples to illustrate how visualizations can reveal or obscure important information.
- **Understanding Spurious Correlations**: This problem highlights the importance of critical thinking when interpreting correlations between variables. Examples of spurious correlations, such as the relationship between per capita cheese consumption and civil engineering doctorates, are presented to caution against drawing causal conclusions from correlations alone.
- **Choosing Appropriate Visualization Types**: This problem emphasizes the importance of selecting the right visualization type to effectively communicate data. The lecture uses examples such as pie charts versus slope graphs to show how different chart types are better suited for different purposes and data types.
- **Impact of Design Choices on Data Perception**: This problem explores how design choices in visualizations, such as axis scaling and color schemes, can influence how viewers interpret the data. The lecture uses examples of production cost graphs with different y-axis scales to illustrate how seemingly minor design decisions can lead to different conclusions.
- **Exploratory Data Analysis**: Maximize insight into data, uncover underlying structure, identify important variables, detect outliers and anomalies, test underlying modeling assumptions, generate hypotheses from data.
- **Data Summary**: Measures of location (mean, median, quartiles, mode) and measures of dispersion or variability (variance, standard deviation, range, skew).
- **Illustrating the Importance of Data Visualization**: Four datasets with identical summary statistics but very different visual representations, demonstrating the importance of visualizing data before drawing conclusions.
- **Creating Scatter Plots**: Using the ‘matplotlib.pyplot.scatter‘ function to generate scatter plots, including parameters for data points, colors, marker styles, and labels.
- **Interpreting Scatter Plots**: Identifying relationships (no relationship, linear relationship, non-linear relationship) and outliers in scatter plots.
- **Creating Subplots**: Using ‘plt.subplots‘ to create grids of subplots and axes, controlling shared x and y limits.
- **Using Scatterplot Matrices**: Visualizing relationships between multiple variables using a scatterplot matrix.
- **Addressing Limitations of Scatter Plots**: Overcoming overprinting in scatter plots using jittering.

- **Using Contour Plots**: Representing 3D surfaces using contour lines in a 2D format to address limitations of 2D scatter plots with high-density data.
- **Creating Line Plots**: Generating line plots and highlighting the importance of sorting data for meaningful visualizations.
- **Creating Histograms**: Generating and interpreting histograms to visualize univariate data distributions, showing center, spread, skew, outliers, and multiple modes.
- **Addressing Limitations of Histograms**: Recognizing limitations of histograms for small datasets and using smoothed density plots as an alternative.
- **Understanding Density Plots**: Estimating continuous functions using kernel functions and bandwidth parameters.
- **Creating Bar Plots**: Generating bar plots to compare average values across different categories.
- **Creating Grouped Bar Charts**: Creating grouped bar charts to compare multiple variables across different categories.
- **Creating Box Plots**: Generating box plots to display relationships between discrete and continuous variables, showing quartiles, range, and outliers.
- **Combining Visualizations**: Combining and coordinating visualizations, such as histograms and scatter plots, using GridSpec for layout.
- **Using ggplot**: Utilizing the grammar of graphics to build plots with high-level abstraction, including aesthetics, geoms, and layers.
- **Using Plotly**: Creating interactive, publication-quality graphs online using Python and the Django framework.
- **Applying Basic Principles of Visualization**: Ensuring data integrity, clarity, appropriate choice of visualization, context, accessibility, interactivity, and visual appeal.
- **Understanding Historical Visualizations**: Examining Playfair's and Minard's visualizations to understand the evolution of data visualization techniques.
- **Choosing Effective Visualizations**: Selecting the best graphic form for data, considering the task, trying different forms, and testing outcomes.
- **Understanding Visual Estimates**: Using the scale of elementary perceptual tasks to create successful charts.
- **Visualizing Transmission**: Using maps to represent the spread of phenomena over geographic areas.
- **Comparing Chart Types**: Evaluating the effectiveness of different chart types (stacked vs. non-stacked area charts) for conveying specific insights.
- **Choosing Appropriate Baselines and Scales**: Determining whether to include 0 as a baseline and using logical and meaningful baselines.
- **Considering Data Aggregation and Normalization**: Highlighting the importance of different data aggregations and normalizations when analyzing time-series data.
- **Addressing Challenges of Varying Ranges**: Dealing with variables that have vastly different ranges in data visualization.
- **Data Quality Problems**: This problem focuses on identifying and handling issues like noise, outliers, missing values, and duplicate data in datasets.

- **Visualizing Data**: This problem involves choosing appropriate visualization techniques to effectively communicate insights from data, while also considering potential biases in visual perception and interpretation.
- **Cognitive Biases in Data Interpretation**: This problem explores how cognitive biases, such as patternicity, storytelling, confirmation bias, availability heuristic, and representativeness heuristic, can lead to flawed interpretations of data and visualizations.
- **Explaining Patterns in Data**: This problem focuses on constructing clear and well-supported explanations of patterns observed in data, requiring a claim, evidence, and reasoning to connect the two.
- **Evaluating Claims Based on Data**: This problem involves critically assessing claims made about data, considering the evidence presented and potential biases or limitations.
- **Asking and Answering Scientific Questions with Data**: This problem involves formulating testable scientific questions that can be addressed using data analysis techniques, and then using data to answer those questions.
- **Understanding and Applying Sample Statistics**: This problem involves understanding the difference between population and sample statistics, and using sample statistics to make inferences about the population.
- **Hypothesis Testing**: This problem involves using statistical methods to test hypotheses about populations based on sample data, including understanding Type I and Type II errors and statistical power.
- **Interpreting Statistical Results**: This problem involves interpreting the results of statistical tests, considering factors such as p-values, statistical power, and the potential for errors.
- **Expressing Uncertainty**: This problem focuses on effectively communicating the uncertainty inherent in data analysis, particularly in survey results and statistical estimations. Methods for quantifying and visualizing uncertainty, such as confidence intervals and error bars, are discussed.
- **Calculating Confidence Intervals**: This problem involves calculating and interpreting confidence intervals for a mean, understanding the role of standard error and Z-scores, and considering corrections for small sample sizes.
- **A/B Testing and Controlled Experiments**: This problem explores the design and analysis of A/B tests and controlled experiments to establish causal relationships between interventions and outcomes. It includes considerations for statistical significance and the challenges of large-scale experimentation.
- **Chi-square Test for Association**: This problem involves using the chi-square test to determine if there is a statistically significant association between two categorical variables, interpreting contingency tables, and understanding the calculation of expected frequencies.
- **Multiple Hypothesis Testing**: This problem addresses the challenges of interpreting p-values and controlling for Type I error when multiple hypotheses are tested simultaneously. Methods like the Bonferroni correction are discussed.
- **Simpson's Paradox**: This problem focuses on understanding and identifying Simpson's paradox, where trends observed in subgroups reverse when the groups are combined. It emphasizes the importance of disaggregating data to avoid misleading conclusions.
- **Causal Inference**: This problem explores the distinction between prediction and causation, and methods for establishing causal relationships from data, including counterfactuals and randomized experiments.
- **Sampling**: This problem focuses on understanding different sampling methods (simple random sampling, stratified sampling, sampling with/without replacement) and their implications for data analysis. It emphasizes the importance of representative samples.

- **Introduction to Clustering**: This slide introduces the concept of clustering as a method for grouping similar objects.
- **Subfields of Machine Learning**: This slide differentiates between supervised and unsupervised learning, with clustering as an example of the latter.
- **What is Clustering?**: This slide defines clustering and its characteristics, emphasizing its role in unsupervised learning.
- **Examples of Clustering**: This section provides examples of clustering applications in various domains, including documents, images, genes, and movies.
- **Personas**: This slide introduces the concept of personas as a way to understand user needs across teams.
- **Clustering Terminology**: This slide defines different types of clustering approaches, including hierarchical, flat, hard, and soft clustering.
- **Similarity/Distance**: This slide introduces the concept of measuring similarity and dissimilarity between data points, focusing on distance measures.
- **Distance Measures for Discrete Data**: This slide introduces the Hamming distance and matching coefficient for measuring dissimilarity between discrete data.
- **K-means Clustering**: This slide introduces the K-means algorithm as a common clustering method.
- **The K-means Algorithm**: This slide details the steps involved in the K-means algorithm.
- **How to Evaluate Clusters**: This slide introduces the problem of evaluating the quality of clustering results.
- **Python Example of K-means Clustering**: This slide provides a Python code example demonstrating K-means clustering using scikit-learn.
- **Interpreting Results**: This slide introduces the problem of interpreting the results of clustering.
- **Quantitative Evaluation**: This slide discusses quantitative evaluation metrics for clustering, highlighting the lack of ground truth in many clustering scenarios.
- **Subjective Evaluation of Clustering**: This slide describes a subjective approach to evaluating clustering results by visually inspecting the clusters and proximity matrices.
- **Objective Evaluation of Clustering**: This slide describes an objective approach to evaluating clustering results using known class labels.
- **Handling Big Data**: This problem focuses on the challenges of processing and managing extremely large datasets, including considerations of volume, velocity, variety, and veracity. The lecture explores solutions such as MapReduce and Hadoop.
- **Understanding MapReduce**: This problem involves learning the MapReduce programming model, including the roles of mapper and reducer functions, the shuffle step, and the overall process of distributed computation. The lecture uses examples such as word counting and matrix multiplication to illustrate the concepts.
- **Comparing Distributed Computing Paradigms**: This problem involves comparing different approaches to distributed computing, such as MapReduce and MPI, highlighting their strengths and weaknesses in terms of complexity, scalability, and fault tolerance.
- **Implementing MapReduce in Python**: This problem focuses on writing MapReduce programs in Python, using libraries like ‘mrjob‘ to simplify the process and leverage Python’s ‘map‘ and ‘reduce‘ functions for intuition.

- **Understanding Relational Databases**: This section covers the fundamental concepts of relational databases, including tables, relations, primary keys, foreign keys, and different types of relationships (one-to-one, one-to-many, many-to-many).
- **Using SQL for Data Manipulation and Querying**: This section focuses on using Structured Query Language (SQL) to interact with relational databases, including creating tables, inserting and deleting data, performing queries, and using joins to combine data from multiple tables.
- **Understanding Database Schema and Instance**: This section differentiates between the database schema (the structure) and the database instance (the data).
- **Comparing Pandas and DBMS**: This section compares the capabilities of the Pandas library in Python with those of a Database Management System (DBMS).
- **Working with SQLite**: This section covers the use of SQLite, a serverless relational database management system.
- **Performing SQL Joins**: This section explains different types of joins (inner, left, right, outer) and how to perform them using SQL and Pandas.
- **Using Aggregation and Grouping in SQL**: This section covers the use of aggregate functions (SUM, COUNT, MIN, MAX, AVG) and the GROUP BY and HAVING clauses to summarize data.
- **Working with Higher-Level Query Languages**: This section introduces higher-level query languages like Pig and Hive, which simplify working with large datasets on Hadoop.
- **Matrix Factorization**: Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction**: Transforming data from a higher-dimensional space to a lower-dimensional space while preserving important information.
- **Singular Value Decomposition (SVD)**: Decomposing a matrix into three simpler matrices (U, S, V<sup>T</sup>) to facilitate dimensionality reduction and matrix reconstruction.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction technique that finds the directions (principal components) that maximize the variance of the data.
- **Identifying and Mitigating Algorithmic Bias**: This problem focuses on identifying and addressing biases present in data science algorithms, including how biases can be inadvertently introduced through training data and how they can perpetuate existing societal inequalities.
- **Ensuring Privacy in Data Science**: This problem explores the challenges of protecting individual privacy when using personal data in data science applications, including the limitations of anonymization techniques and the ethical considerations of data usage.
- **Balancing Accuracy and Interpretability in Machine Learning Models**: This problem highlights the trade-off between the accuracy of a machine learning model and its interpretability, particularly in high-stakes domains like healthcare, where understanding the model's decision-making process is crucial for trust and accountability.
- **Defining and Measuring Fairness in Algorithmic Outcomes**: This problem delves into the complexities of defining and measuring fairness in the outcomes produced by data science algorithms, exploring different approaches to fairness (e.g., group fairness, individual fairness) and their limitations.

# 3 Algorithm Solutions

- **isPalindrome(string)**: This function takes a string as input and returns True if the string is a palindrome (reads the same forwards and backward), and False otherwise. The comparison is case-insensitive.
- **Ensemble Methods**: Combining multiple models to improve predictive performance. The Netflix Prize winning approach used an ensemble of 100+ models.
- **Cox Proportional Hazards Model**: A statistical model used to analyze time-to-event data, such as the time until the onset of sepsis.
- **Reinforcement Learning**: A machine learning technique used to learn optimal sequences of decision rules, as in personalized autism treatments.
- **Recursive Power Function**: This function calculates $x^n$ recursively. If $n \leq 0$, it returns 1. Otherwise, it recursively calculates $x^{n//2}$, squares the result, and multiplies by $x$ if $n$ is odd.
- **Call by Assignment**: In Python, function parameters are passed by assignment. This means that a reference to the object is passed, not a copy of the object itself. For mutable objects, this behaves like call by reference; for immutable objects, it behaves like call by value.
- **String Upper and Lower**: These methods convert a string to uppercase and lowercase, respectively. For example, "'hello".upper()' returns "'HELLO"' and "'Hello".lower()' returns "'hello"'.
- **File Reading**: This algorithm opens a file, reads its contents line by line, and processes each line. It uses 'open()' to open the file, a 'for' loop to iterate through lines, and 'close()' to close the file.
- **CSV File Reading**: This algorithm opens a CSV file, reads it using the 'csv' module, and processes each row. It uses 'csv.reader()' to parse the file and a 'for' loop to iterate through rows.
- **List Append**: This method adds an element to the end of a list. 'list.append(element)'
- **List Insert**: This method inserts an element at a specified index in a list. 'list.insert(index, element)'
- **List Extend**: This method extends a list by appending all items from another iterable (like another list). 'list.extend(iterable)'
- **List Index**: This method returns the index of the first occurrence of a specified element in a list. 'list.index(element)'
- **List Count**: This method returns the number of times a specified element appears in a list. 'list.count(element)'
- **List Remove**: This method removes the first occurrence of a specified element from a list. 'list.remove(element)'
- **List Reverse**: This method reverses the order of elements in a list in-place. 'list.reverse()'
- **List Sort**: This method sorts the elements of a list in-place. 'list.sort()' or 'list.sort(key=some$_function$)'$forcustomsorting$ $Thisinvolvesaccessingavalueinadictionaryusingitskey.$'$dictionary[key]$'
- **Dictionary Deletion**: This involves removing a key-value pair from a dictionary using the 'del' keyword. 'del dictionary[key]'
- **Dictionary Clear**: This method removes all items from a dictionary. 'dictionary.clear()'
- **Dictionary Keys**: This method returns a view object containing the keys of a dictionary. 'dictionary.keys()'
- **Dictionary Values**: This method returns a view object containing the values of a dictionary. 'dictionary.values()'
- **Dictionary Items**: This method returns a view object containing the key-value pairs of a dictionary as tuples. 'dictionary.items()'

- **‘pd.read_csv‘**: This function reads data from a CSV file into a Pandas DataFrame.
- **‘pd.DataFrame‘**: This function creates a Pandas DataFrame from various data structures, such as dictionaries or lists.
- **‘data.describe()‘**: This method generates descriptive statistics (count, mean, std, min, max, etc.) for numerical columns in a Pandas DataFrame.
- **‘data.iloc[]‘**: This attribute accesses data in a Pandas DataFrame using integer-based indexing.
- **‘data.loc[]‘**: This attribute accesses data in a Pandas DataFrame using label-based indexing.
- **‘data.rename()‘**: This method renames the row indexes or column names of a Pandas DataFrame.
- **Regular Expression Matching**: The ‘re‘ module in Python provides functions like ‘re.compile()‘, ‘re.match()‘, ‘re.search()‘, ‘re.findall()‘, and ‘re.finditer()‘ to create and apply regular expression patterns to strings for pattern matching and extraction.
- **Webpage Parsing with Beautiful Soup**: The ‘BeautifulSoup‘ library in Python allows for parsing HTML and XML documents. Functions like ‘soup.find$_all()‘canbeusedtolocatespecifictags(e.g.,‘ < a > ‘,‘ < p > ‘)andextracttheirtextcontentorattributes(e.g.,‘href‘)$.**TF-IDF Calculation** : $TF - IDFiscalculatedas$TF $\times IDF$, where $TF$ is the term frequency (number of times a word appears in a document divided by the total number of words in the document) and $IDF = ln(\frac{N}{1+n_t})$, where $N$ is the total number of documents and $n_t$ is the number of documents containing the term.
- **Document-Term Matrix Construction**: A document-term matrix is created by preprocessing text data (removing punctuation, converting to lowercase, identifying unique words), counting word occurrences in each document, and organizing these counts into a matrix where rows represent documents and columns represent unique words.
- **‘data_zillow.columns = [...]‘**: Renames columns in a pandas DataFrame.
- **‘data_zillow.sort_values(...)‘**: Sorts a pandas DataFrame by specified column(s).
- **‘data_zillow.sort_index(...)‘**: Sorts a pandas DataFrame by index (row or column).
- **‘data_zillow.describe()‘**: Generates descriptive statistics for a pandas DataFrame.
- **‘data_zillow[’SOLD’] = ..‘**: Adds a new column to a pandas DataFrame.
- **‘data_zillow[’PRICE/SQ FT (\$)’] = ..‘**: Adds a calculated column to a pandas DataFrame.
- **‘data_zillow.loc[20] = ..‘**: Adds a new row to a pandas DataFrame using ‘.loc‘.
- **‘data_zillow = data_zillow.append(..., ignore_index=True)‘**: Adds a new row to a pandas DataFrame using ‘.append‘.
- **‘del data_zillow[’INDEX’]‘**: Deletes a column from a pandas DataFrame.
- **‘data_zillow[data_zillow.BEDS == 3]‘**: Filters a pandas DataFrame based on a condition.
- **‘data_zillow[[’ZIP’,’YEAR’]]‘**: Selects specific columns from a pandas DataFrame.
- **‘data_zillow.sample(10)‘**: Randomly samples rows from a pandas DataFrame.
- **‘df_employee.duplicated()‘**: Identifies duplicate rows in a pandas DataFrame.
- **‘df_employee.loc[df_employee.duplicated()]‘**: Selects duplicate rows using boolean indexing.
- **‘df_employee.drop_duplicates()‘**: Removes duplicate rows from a pandas DataFrame.
- **‘df_weather.replace(...)‘**: Replaces values in a pandas DataFrame.

- **‘df_weather.replace(..., inplace=True)‘**: Replaces values in a pandas DataFrame in place.
- **‘df_grades.isnull().sum()‘**: Counts missing values (NaN) in a pandas DataFrame.
- **‘df_grades[df_grades.Test2.isnull()]‘**: Filters rows with missing values.
- **‘df_grades.iloc[8, 4] = 50.0‘**: Assigns a value to a specific cell in a DataFrame.
- **‘df_grades.fillna(0)‘**: Fills missing values with 0.
- **‘df_weather.fillna(method=’ffill’)‘**: Fills missing values using forward fill.
- **‘df_weather.fillna(method=’bfill’)‘**: Fills missing values using backward fill.
- **‘df_weather.interpolate()‘**: Interpolates missing values.
- **‘df_weather.dropna()‘**: Removes rows with missing values.
- **‘df_final[’Normalized’] = ...‘**: Normalizes a column in a DataFrame.
- **‘pd.cut(df_final.Grade, bins)‘**: Bins numerical data into categories.
- **‘pd.cut(df_final.Grade, bins, labels=bin_names)‘**: Bins numerical data with custom labels.
- **‘df_final[’Final Grade’] = ...‘**: Adds a new column with letter grades.
- **‘df_stock.groupby(’Symbol’)‘**: Groups a DataFrame by a column.
- **‘grouped_stock.max()‘**: Applies the ‘max‘ function to grouped data.
- **‘grouped_stock.mean()‘**: Applies the ‘mean‘ function to grouped data.
- **‘pd.concat([df_A, df_B])‘**: Concatenates DataFrames vertically.
- **‘pd.concat([df_A, df_B], ignore_index=True)‘**: Concatenates DataFrames vertically, resetting the index.
- **‘pd.concat([df_no_exam, df_exam_1], axis=1)‘**: Concatenates DataFrames horizontally.
- **‘mask = \textasciitilde{}df_grades.columns.duplicated()‘**: Creates a boolean mask for duplicate columns.
- **‘pd.merge(df_student, df_gpa)‘**: Performs an inner merge of DataFrames.
- **‘pd.merge(df_student, df_gpa, how=’left’)‘**: Performs a left merge of DataFrames.
- **‘pd.merge(df_student, df_gpa, how=’right’)‘**: Performs a right merge of DataFrames.
- **‘pd.merge(df_student, df_gpa, how=’outer’)‘**: Performs an outer merge of DataFrames.
- **‘pd.merge(df_student, df_gpa, left_on=’..’, right_on=’..’)‘**: Merges DataFrames with mismatched column names.
- **‘pd.merge(df_student, df_gpa, left_index=True, right_on=’..’)‘**: Merges using the index of one DataFrame.
- **‘pd.merge(df_student, df_gpa, left_index=True, right_index=True)‘**: Merges using the indices of both DataFrames.
- **‘wc -l file1.dat‘**: Counts lines in a file.
- **‘grep pattern file‘**: Searches for a pattern in a file.
- **‘sed ’s/,/\t/g’‘**: Substitutes commas with tabs.

- **‘awk ’{print \$1}’‘**: Prints the first field of each record.
- **‘awk ’{print NR “,“ \$0}’‘**: Adds line numbers to a file.
- **Linear Regression**: $y = \beta_0 + \beta_1 x + \epsilon$
- **Scatter Plot**: A graphical representation of data points on a two-dimensional plane, showing the relationship between two variables.
- **Pie Chart**: A circular statistical graphic, which is divided into slices to illustrate numerical proportion.
- **Slope Chart**: A chart that shows the change in values over time for different categories.
- **Line Graph**: A type of chart which displays information as a series of data points called ’markers’ connected by straight line segments.
- **Bar Graph**: A chart that uses bars to represent data, comparing different categories or groups.
- **Choropleth Map**: A thematic map in which areas are shaded or patterned in proportion to the measurement of the statistical variable being displayed on the map.
- **Dot Map**: A map that uses dots to represent the location of data points.
- **Proportional Symbol Map**: A type of map that uses symbols of varying sizes to represent the magnitude of a phenomenon at different locations.
- **Isarithmic Map**: A thematic map with lines that connect points of equal value.
- **Principal Component Analysis (PCA)**: A dimensionality reduction technique that transforms a dataset into a new set of uncorrelated variables (principal components) that capture the maximum variance in the data.
- **Kernel Density Estimation (KDE)**: A non-parametric method for estimating the probability density function of a random variable. The formula is given by $\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^{n} K\left(\frac{x - x^{(i)}}{h}\right)$, where $\hat{f}(x)$ is the estimated density at point x, n is the number of data points, h is the bandwidth, and K is the kernel function.
- **Central Limit Theorem**: This theorem states that the distribution of the sample mean approaches a normal distribution as the sample size increases, regardless of the shape of the population distribution. This allows for the use of normal or t-distributions in hypothesis testing for sufficiently large sample sizes.
- **Standard Error**: $SE = \frac{\sigma}{\sqrt{n}}$
- **Confidence Interval Calculation**: (Point value) $\pm$ Z $\times$ standard error
- **Chi-square Statistic**: $\chi^2 = \sum_{i,j} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$
- **Probability of at least one false positive in m tests**: $1 - (1 - \alpha)^m$
- **Euclidean Distance**: $d_E(x, y) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}$
- **Hamming Distance**: Counts the number of bits that differ between two binary vectors.
- **Matching Coefficient**: Hamming distance normalized by the number of attributes/bits.

- **Purity**: $\text{purity}(C, G) = \frac{1}{N} \sum_{i=1}^{K} \max_j N_j \in G_i$
- **MapReduce**: A programming model for processing large datasets on clusters of commodity hardware, involving mapper and reducer functions.
- **Mapper**: A function that transforms data records into key-value pairs.
- **Reducer**: A function that combines and simplifies intermediate data appropriately.
- **Shuffle**: A step that uses a hash function to group pairs with the same key together on the same machine.
- **Python map**: Applies a function to each element of an iterable.
- **Python reduce**: Iteratively applies a function to two elements of an iterable until a single result is obtained.
- **SQL Join**: This operation merges multiple tables into a single relation based on columns from each table that specify which rows are kept.
- **SQL Aggregation**: This operation calculates summary values on a selection using aggregate functions such as SUM, COUNT, MIN, MAX, and AVG.
- **SQL Grouping**: This operation groups rows with the same values in specified columns, allowing for aggregate functions to be applied to each group.
- **Singular Value Decomposition (SVD)**: $A = USV^T$, where $A$ is the input matrix, $U$ and $V$ are orthogonal matrices, and $S$ is a diagonal matrix containing singular values.
- **Collaborative Filtering**: A recommender system algorithm that infers missing ratings based on the ratings of similar users.
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of predictions in recommender systems. It is calculated as $\sqrt{\frac{1}{|R|} \sum_{(u,i) \in R} (r_{ui} - \hat{r}_{ui})^2}$, where $r_{ui}$ is the actual rating and $\hat{r}_{ui}$ is the predicted rating.
- **Fairness through Blindness**: Ignore irrelevant/protected attributes. However, high accuracy prediction of an attribute is possible even without directly observing it.
- **Group Fairness**: $P(\text{outcome } o|S) = P(\text{outcome } o|T)$, where $S$ and $T$ are two groups (e.g., minority and majority). This is not a strong enough notion of fairness.
- **Individual Fairness**: Treat similar individuals similarly for the purpose of the classification task, with outcomes similarly distributed.